

Défense Périphérique, Traçabilité et Chiffrement (Linux)

- Déployer une politique de micro-segmentation réseau locale (HBF) via Netfilter/UFW.
- Concevoir une architecture de centralisation sécurisée des journaux (Syslog/TLS).
- Maîtriser la cryptographie des disques de données (LUKS) et sa structure interne.
- Durcir la synchronisation temporelle (NTP/Chrony) pour garantir l'intégrité légale des logs.

1. Micro-segmentation et Filtrage Réseau Local (HBF)

Le paradigme de la sécurité périmétrique (un pare-feu matériel en bordure de réseau et une confiance aveugle à l'intérieur du LAN) est obsolète. L'ANSSI et l'approche *Zero Trust* exigent le déploiement d'un **Host-Based Firewall (HBF)** sur chaque système d'exploitation.

1.1 L'architecture Netfilter et Nftables

Sous Linux, le filtrage réseau n'est pas un logiciel additionnel, c'est un composant natif du noyau : **Netfilter**. Netfilter intercepte les paquets réseau à différents "Hooks" (points d'ancrage) dans la pile TCP/IP du noyau (PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING).

- **Iptables (L'ancien standard)** : Historiquement, les administrateurs utilisaient **iptables** pour parler à Netfilter. Ses limites (évaluation séquentielle très lente, syntaxe complexe, IPv4/IPv6 séparés) l'ont rendu obsolète.
- **Nftables (Le nouveau standard)** : Depuis Debian 10 / RHEL 8, **nftables** remplace iptables. Il utilise une machine virtuelle intégrée au noyau, regroupe IPv4 et IPv6, et offre des performances drastiquement supérieures grâce aux structures de données (Sets/Maps).

1.2 UFW (Uncomplicated Firewall) : L'implémentation opérationnelle

Écrire des règles **nftables** brutes est fastidieux. Les distributions proposent des surcouches d'abstraction. Sur l'écosystème Debian/Ubuntu, **UFW** est le standard. Il traduit des commandes simples en règles Netfilter complexes.

Configuration d'un Bastion (La politique du Default Deny) : La règle d'or consiste à fermer toutes les portes, puis à percer des ouvertures chirurgicales (ports et adresses IP sources).

1. # 1. Activation de la journalisation pour tracer les paquets bloqués (mode moyen)

2. `sudo ufw logging medium`
- 3.
4. # 2. Politique par défaut stricte
5. `sudo ufw default deny incoming`
6. `sudo ufw default allow outgoing`
- 7.
8. # 3. Exceptions chirurgicales (Limitation d'accès par IP et sous-réseau)
9. # Exemple : SSH uniquement depuis le VLAN d'administration
10. `sudo ufw allow from 192.168.10.0/24 to any port 22 proto tcp`
- 11.
12. # Exemple : Limitation du taux de connexion (Anti-Brute Force intégré)
13. # Bloque l'IP si elle initie plus de 6 connexions en 30 secondes
14. `sudo ufw limit ssh`
- 15.
16. # 4. Activation (charge les règles dans Netfilter)
17. `sudo ufw enable`

2. Centralisation et Inviolabilité des Journaux

Dans le cadre d'une compromission, la première action d'un attaquant obtenant les privilèges `root` est l'effacement de ses traces (`rm -rf /var/log/*` ou `echo "" > /var/log/auth.log`). Un journal stocké localement n'a aucune valeur juridique s'il n'a pas été externalisé en temps réel.

2.1 Le standard Syslog et les "Facilities"

Les journaux Linux sont gérés par le standard **Syslog** (porté aujourd'hui par le démon `rsyslog` ou `journald`). Chaque log est qualifié par deux étiquettes :

- **Facility (La source)** : `auth` (authentification), `kern` (noyau), `mail`, `cron`, etc.
- **Severity (La gravité)** : `emerg` (0), `alert` (1), `crit` (2), `err` (3), `warning` (4), `notice` (5), `info` (6), `debug` (7).

2.2 Externalisation sécurisée (Exigences ANSSI)

L'ANSSI interdit l'exportation des logs en clair (via UDP 514 ou TCP 514 simple), car un attaquant en position de *Man-in-the-Middle* pourrait lire les alertes ou injecter de faux journaux.

Configuration `rsyslog` vers un SIEM avec chiffrement TLS : Dans le fichier `/etc/rsyslog.conf` ou `/etc/rsyslog.d/99-siem.conf` :

```
18. # 1. Chargement du module TCP
19. $ModLoad imtcp
20.
21. # 2. Configuration des certificats TLS
22. $DefaultNetstreamDriverCAFile /etc/ssl/certs/ca-entreprise.crt
23. $DefaultNetstreamDriverCertFile /etc/ssl/certs/serveur-linux.crt
24. $DefaultNetstreamDriverKeyFile /etc/ssl/private/serveur-linux.key
25.
26. # 3. Obligation d'utiliser le pilote TLS
27. $DefaultNetstreamDriver gtls
28. $ActionSendStreamDriverMode 1 # Authentification requise
29. $ActionSendStreamDriverAuthMode x509/name
30.
31. # 4. Règle d'exportation : Tout envoyer (*.*) vers le SIEM sur le port 6514
32. *.* @@logs.entreprise.local:6514
```

(Note : L'utilisation du double @@ spécifie le protocole TCP. Un seul @ spécifie UDP).

3. Cryptographie de la "Donnée au Repos" : Architecture LUKS

Le durcissement réseau et logique est inutile face à un accès physique. Si un serveur est volé, ou si une machine virtuelle est copiée (Snapshot `.vmdk`), l'attaquant contourne l'OS et lit directement les blocs de la base de données.

3.1 Anatomie de LUKS (Linux Unified Key Setup)

LUKS est le standard industriel de chiffrement de disque sous Linux (équivalent de BitLocker). Il opère au niveau bloc (sous le système de fichiers LVM ou Ext4) via le sous-système natif du noyau `dm-crypt`.

Le mécanisme de la clé maître :

1. Lors du chiffrement, LUKS génère une clé aléatoire incassable : la **Volume Master Key (VMK)**. C'est elle qui chiffre réellement les données avec l'algorithme AES-XTS.
2. L'utilisateur ne voit jamais cette VMK.
3. La phrase secrète tapée par l'administrateur (ou la clé contenue dans le TPM) est utilisée pour chiffrer la VMK.
4. LUKS dispose de **8 emplacements (Key Slots)** dans l'en-tête du disque. Cela permet à 8 administrateurs d'avoir des mots de passe différents capables de déverrouiller la

même VMK, sans avoir à rechiffrer les données.

3.2 L'En-tête LUKS

Toute la cryptographie repose sur l'en-tête (LUKS Header) situé dans les deux premiers mégaoctets de la partition. **Si ce header est corrompu (secteur défectueux) ou effacé, les données sont définitivement perdues**, même si vous possédez le mot de passe. La sauvegarde du header est une procédure de secours vitale en production :

```
33. sudo cryptsetup luksHeaderBackup /dev/sdb1 --header-backup-file  
    /root/sdb1-header.img
```

3.3 Commandes de cycle de vie LUKS

34. **Initialisation sécurisée (ANSSI : AES 256 bits minimum) :**

```
sudo cryptsetup luksFormat -c aes-xts-plain64 -s 512 -h sha512 -y /dev/sdb1
```

-

35. **Ouverture du coffre (Mapping) :**

```
sudo cryptsetup luksOpen /dev/sdb1 mon_disque_secret
```

- (Le disque déchiffré est alors accessible de manière transparente via `/dev/mapper/mon_disque_secret`).

36. **Gestion de la flotte (Ajout d'une clé de secours) :**

```
sudo cryptsetup luksAddKey /dev/sdb1
```

-

4. Intégrité de la Chronologie : Durcissement NTP

Le temps (NTP - Network Time Protocol) n'est pas qu'une question de confort, c'est un vecteur d'attaque critique et un pilier cryptographique.

4.1 Les impacts d'une horloge manipulée

Un attaquant modifiant l'horloge système peut :

1. **Corrompre l'investigation (Forensics) :** Placer ses logs d'attaque dans le passé ou dans le futur pour tromper les analystes SOC.
2. **Briser la cryptographie :** Faire valider des certificats TLS expirés ou révoqués.
3. **Détruire l'authentification :** Les tickets Kerberos (Active Directory) ont une durée de vie stricte. Si le serveur Linux est décalé de plus de 5 minutes avec le Contrôleur de Domaine, toute l'authentification réseau échoue (Déni de service).

4.2 Configuration sécurisée avec Chrony

Le démon `chronyd` est le remplaçant moderne de `ntpd`. Il doit être durci dans `/etc/chrony/chrony.conf` :

```
37. # 1. Utiliser uniquement des serveurs internes validés (pas de pool public)
38. server ntp1.entreprise.local iburst
39. server ntp2.entreprise.local iburst
40.
41. # 2. Interdire au serveur Linux de distribuer l'heure aux autres (il n'est que client)
42. port 0
43.
44. # 3. Limiter les sauts de temps (Step)
45. # N'autoriser un saut brut que sur les 3 premières requêtes au démarrage.
46. # Ensuite, la correction doit se faire de manière lente et lissée (Slew)
47. makestep 1.0 3
48.
49. # 4. Bloquer les commandes de gestion à distance
50. cmdport 0
```

51.

•